SolarBI Pascual

Power BI y Grafana, gobernanza de datos,
automatización ETL e Internet de las Cosas

Estudiante	Santiago Velásquez Serna
Documento	1038414022
Asignatura	Inteligencia de Negocios
Grupo	50
Semestre	2026-II
Docente	Ramiro Grisales Montoya
Modalidad	Trabajo individual
Fecha	9 de octubre de 2026

Repositorio: https://github.com/savelasquezs/Grafana_power_bi

Parte A · Consulta

Las respuestas se aplican al caso SolarBI. Las referencias numeradas se presentan al final.

D1. ¿Qué es Grafana?

Grafana es una plataforma para consultar fuentes de datos y convertir sus resultados en observabilidad visual. Un *data source* define la conexión; cada *panel* ejecuta una consulta y dibuja una visualización; un *dashboard* organiza paneles; las variables parametrizan consultas; y *alerting* evalúa reglas para notificar condiciones. Grafana OSS se instala y administra localmente. Grafana Cloud es un servicio gestionado que agrega operación, escalado e integraciones. En SolarBI se usa localmente sobre PostgreSQL. [1]

D2. Power BI frente a Grafana

Power BI prioriza el análisis de negocio y el modelo semántico; Grafana prioriza seguimiento operativo y consulta directa del origen. En SolarBI son complementarios: Power BI resume rendimiento y ahorro; Grafana sigue telemetría y calidad.

Criterio	Power BI	Grafana
Propósito	Análisis y decisiones	Monitoreo operativo
Usuario	Analista y directivo	Operador e ingeniería
Datos	Import o DirectQuery	Consulta al data source
Semántica	Modelo tabular y DAX	SQL del origen
Actualización	Refresh del modelo	Auto-refresh del tablero
Alertas	Servicio/Fabric	Reglas de alerta
Costo	Desktop + licencias de servicio	OSS local o Cloud
Acceso	Workspace, RLS	Organización, roles y carpetas

D3. Dashboards operativo, analítico y estratégico

El operativo responde al presente y a excepciones; el analítico explica tendencias y causas; el estratégico resume metas para dirección. En Grafana publicaría disponibilidad, alarmas y porcentaje válido por su refresco frecuente. En Power BI publicaría energía, yield, Performance Ratio y ahorro por su modelo DAX y navegación analítica. Un resumen mensual de todos ellos sería estratégico en Power BI; la telemetría de cinco minutos permanece operativa en Grafana.

D4. Import, DirectQuery y actualizaciones

Import copia datos al modelo, ofrece máxima interactividad y requiere actualización. DirectQuery mantiene los datos en PostgreSQL y consulta al interactuar, con latencia y límites del origen. Para tres días de SolarBI usaría Import por volumen pequeño, rapidez y medidas DAX; una actualización diaria basta. Grafana puede reconsultar PostgreSQL cada cinco minutos sin construir un modelo importado. Si la planta creciera y exigiera muy baja latencia, evaluaría DirectQuery o un modelo híbrido. [2][3]

D5. Variables de Grafana y slicers de Power BI

Una variable de Grafana sustituye valores dentro del SQL antes de enviarlo al origen; un slicer filtra el contexto del modelo semántico de Power BI. En SolarBI la variable *dispositivo* obtiene clave y nombre desde *dwh.dim_dispositivo*. Cada panel añade *dispositivo_key = ${dispositivo:raw}*, de modo que al seleccionar un inversor se vuelven a ejecutar las consultas solo para ese equipo. [4]

G1. Gobernanza, gestión y calidad

La gobernanza define decisiones, responsabilidades, políticas y controles sobre los datos. La gestión ejecuta el ciclo de vida: integración, almacenamiento, seguridad, metadatos y uso. La calidad mide si los datos son aptos para el propósito mediante reglas como completitud, unicidad y validez. En SolarBI la gobernanza decide contrato, responsables y umbrales; el ETL gestiona el flujo; y Silver evidencia la calidad. El enfoque coincide con la separación de capacidades propuesta por DAMA-DMBOK. [5]

G2. Roles de datos

El *data owner* responde por propósito y reglas: en SolarBI equivale al product owner. El *data steward* mantiene definiciones, calidad y catálogo: lo comparten modelador y analista BI. El *data custodian* opera plataformas, accesos y respaldos: corresponde principalmente al data engineer. Como trabajo individual, una persona ejecutó todos los roles, pero se conservaron separados conceptualmente para no confundir autorización, definición y operación.

G3. Contrato de datos

Debe fijar campos y tipos, unidades, frecuencia de cinco minutos, zona horaria, productor, responsable, reglas de faltantes, duplicados y rangos, y comportamiento ante cambios. Si la fuente renombra *p_ac_kw* sin aviso, el ETL puede fallar o producir nulos; los agregados Gold y ambos tableros quedarían incompletos. El cambio debe versionarse, validarse en desarrollo y comunicarse antes de modificar producción.

G4. Fuente única de verdad y catálogo de KPI

La fuente única de verdad es el conjunto gobernado desde el que todos calculan indicadores. El catálogo registra definición, fórmula, unidad, linaje, responsable y frecuencia. Si Power BI calcula un KPI en DAX y Grafana otro en SQL con filtros distintos, aparecerán cifras contradictorias. Por eso la base diaria y las definiciones se centralizan y las expresiones se revisan contra el mismo contrato.

KPI	Fórmula	Unidad	Fuente	Responsable	Frecuencia
Yield	Energía / 5 kWp	kWh/kWp	fact_energia_dia	Analista BI	Diaria
Datos válidos	válidas / totales × 100	%	fact_energia_dia	Data steward	Diaria

G5. Ley 1581 de 2012

La telemetría técnica de un inversor no identifica por sí sola a una persona, por lo que el conjunto actual no es dato personal. Si se agrega el operador que atendió cada alarma, ya existiría información asociada a una persona identificable. SolarBI aplicaría finalidad, acceso mínimo, seguridad, retención definida y trazabilidad; para análisis usaría un identificador interno y mostraría nombres solo a usuarios autorizados. [6]

E1. ETL, ELT y capas

ETL transforma antes de cargar; ELT carga primero y transforma dentro de la plataforma. En SolarBI, Bronze conserva CSV crudo, Silver valida y tipa lecturas y Gold agrega energía y calidad diaria. El flujo implementado es ETL porque Python limpia antes de insertar Silver y genera Gold. Power Query sí es una herramienta ETL: extrae de PostgreSQL, transforma con pasos reproducibles y carga el modelo de Power BI, aunque aquí se mantuvo mínimo porque la calidad ya quedó resuelta aguas arriba.

E2. Carga completa, incremental e idempotencia

La carga completa reemplaza o reprocesa todo el conjunto; la incremental procesa solo registros nuevos o cambiados. Idempotencia significa que repetir la misma entrada deja el mismo estado final. En SolarBI las claves primarias de *silver.lectura_5min* y *dwh.fact_energia_dia*, junto con *INSERT ... ON CONFLICT DO UPDATE*, evitan duplicar filas. Dos ejecuciones consecutivas conservaron 832 lecturas Silver y tres filas diarias.

E3. Automatización del flujo

Para esta entrega usaría cron o el Programador de tareas por simplicidad. Airflow es mejor cuando crecen dependencias, reintentos y observabilidad. La actualización programada de Power BI refresca el modelo publicado, pero no reemplaza la limpieza ni la carga de PostgreSQL. [7]

Opción	Qué automatiza	Cuándo conviene
cron / Programador	Un comando a una hora	Flujo pequeño y estable
Apache Airflow	DAG, dependencias, reintentos	Varios procesos y seguimiento
Power BI refresh	Modelo semántico publicado	Datos ya preparados para BI

E4. Actualización incremental en Power BI

RangeStart y RangeEnd son parámetros DateTime reservados que filtran la tabla. Al publicar, el servicio crea particiones históricas y refresca solo la ventana reciente. Vale la pena con años de telemetría, ventanas de actualización largas o alto costo de consulta. Para solo tres días y tres filas Gold, recargar todo es más simple y verificable; añadir particiones sería sobreingeniería. [8]

I1. Arquitectura IoT por capas

El inversor mide potencia, irradiancia y temperatura. Un gateway edge normaliza la lectura; la red o un broker la transporta; el almacenamiento conserva el evento en Bronze, luego el ETL lo valida en Silver y agrega el día en Gold; finalmente Grafana consulta Silver/Gold y Power BI consulta Gold. Flujo: **sensor** → **gateway** → **broker/red** → **Bronze** → **Silver** → **Gold** → **visual**. Cada transición conserva trazabilidad hacia la lectura original.

I2. MQTT

MQTT usa publish/subscribe: un dispositivo publica mensajes en un *topic*, el broker los distribuye y los consumidores se suscriben. QoS 0 entrega como máximo una vez; QoS 1, al menos una vez; QoS 2, exactamente una vez mediante más intercambio. Un topic adecuado sería *pascualbravo/solar/sede1/inversor1/p_ac_kw*. Para telemetría de cinco minutos usaría QoS 1 y deduplicaría en Silver. [9]

I3. Modbus, MQTT y OPC UA

En una planta solar esperaría Modbus TCP/RTU dentro del campo para leer inversores y medidores; un gateway podría publicar por MQTT hacia la plataforma. OPC UA sería útil cuando se necesiten modelos semánticos industriales, descubrimiento y seguridad integrada.

Protocolo	Modelo	Uso	Datos	Seguridad
Modbus RTU/TCP	Maestro/servidor	Equipos de campo	Registros	Debe protegerse con red
MQTT	Publish/subscribe	Telemetría a broker	Payload libre	TLS, autenticación y ACL
OPC UA	Cliente/servidor y pub/sub	Interoperabilidad industrial	Modelo tipado	Cifrado, firma y certificados

I4. Batch frente a streaming

Batch agrupa lecturas y es eficiente para cierres diarios, pero pierde inmediatez y detalle. Streaming conserva reacción rápida, aunque exige más operación y control de eventos tardíos. La agregación diaria reduce 288 intervalos a una fila: facilita yield y ahorro, pero oculta caídas breves. Grafana atiende mejor los cinco minutos; Power BI atiende mejor el análisis diario y ejecutivo. El diseño conserva ambas granularidades para no sacrificar trazabilidad.

V1. Git, GitHub, PBIX y PBIP

Git registra versiones localmente; GitHub aloja y facilita colaboración. Un repositorio agrupa el proyecto; un commit fija un cambio; una branch aísla trabajo; un pull request propone revisión. PBIX es binario: Git no muestra diferencias internas útiles y cada cambio reemplaza el archivo. PBIP separa informe y modelo en archivos de texto, aptos para diff y revisión. Grafana se versiona exportando su dashboard JSON. En esta entrega se conservó el PBIX funcional y el JSON legible. [10]

Parte B · Práctica guiada

El flujo se ejecutó con un único punto de entrada y ambos tableros quedaron conectados a la base **solarbi** del mismo proyecto Neon. No se incluyeron credenciales en el repositorio.

Paso 1 · Bronze

Paso 1 · Bronze: telemetría sin modificar

ts | dispositivo_id | p_ac_kw | irradiancia_wm2 | temp_modulo_c

2026-10-05 00:00:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:05:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:10:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:15:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:20:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:25:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:30:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:35:00 | 1 | 0.0 | 0.0 | 22.0

El simulador produjo tres días de lecturas cada cinco minutos e inyectó faltantes, duplicados y valores fuera de rango. El CSV crudo se conserva sin modificaciones.

Paso 2 · Silver

Paso 2 · Silver: calidad de datos

Filas leídas: 884

Rechazadas por faltantes: 17

Rechazadas por duplicado: 20

Rechazadas por rango: 15

Filas válidas: 832

Porcentaje válido: 94.12 %

La limpieza aplicó las tres reglas mínimas: faltantes, duplicados y rango físico. El reporte deja trazables los rechazos y el porcentaje válido.

Paso 3 · Gold e idempotencia

Paso 3 · PostgreSQL e idempotencia

Primera ejecución: silver.lectura_5min = 832; dwh.fact_energia_dia = 3.

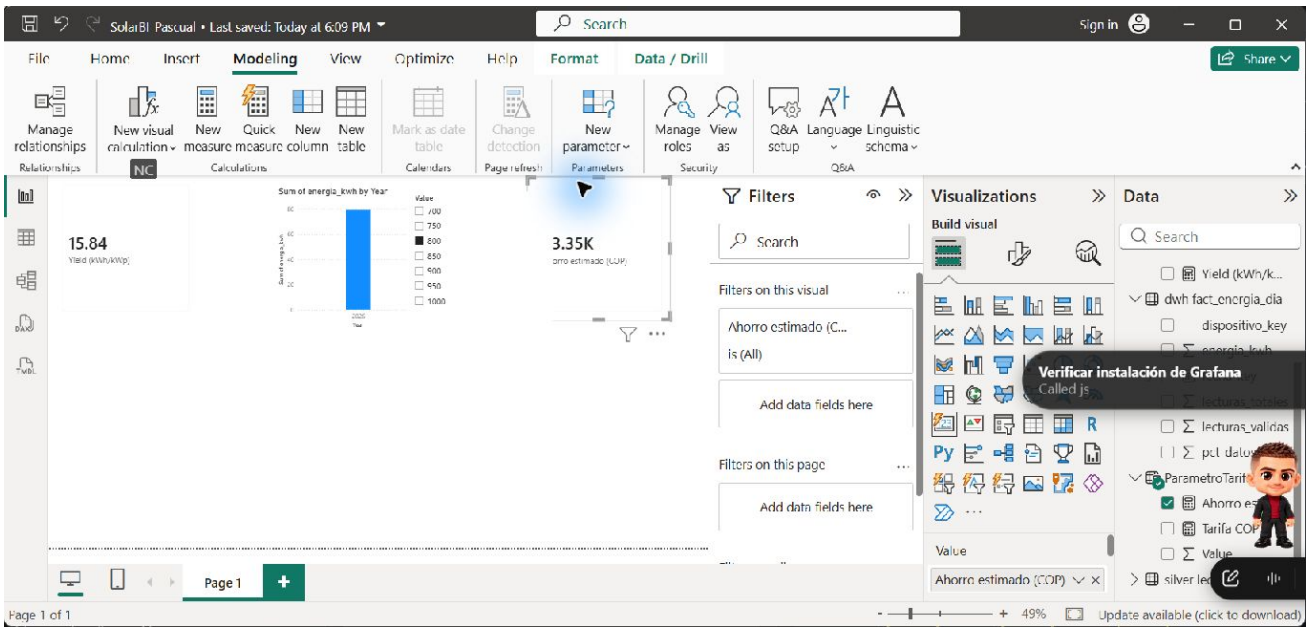
Segunda ejecución: silver.lectura_5min = 832; dwh.fact_energia_dia = 3.

Los conteos permanecieron iguales gracias a claves primarias y UPSERT.

Programación diaria propuesta: 0 0 * * * python etl/run_etl.py

La segunda ejecución mantuvo los mismos conteos. Las claves y UPSERT permiten repetir el proceso sin duplicar lecturas ni resúmenes diarios.

Paso 4 · Power BI



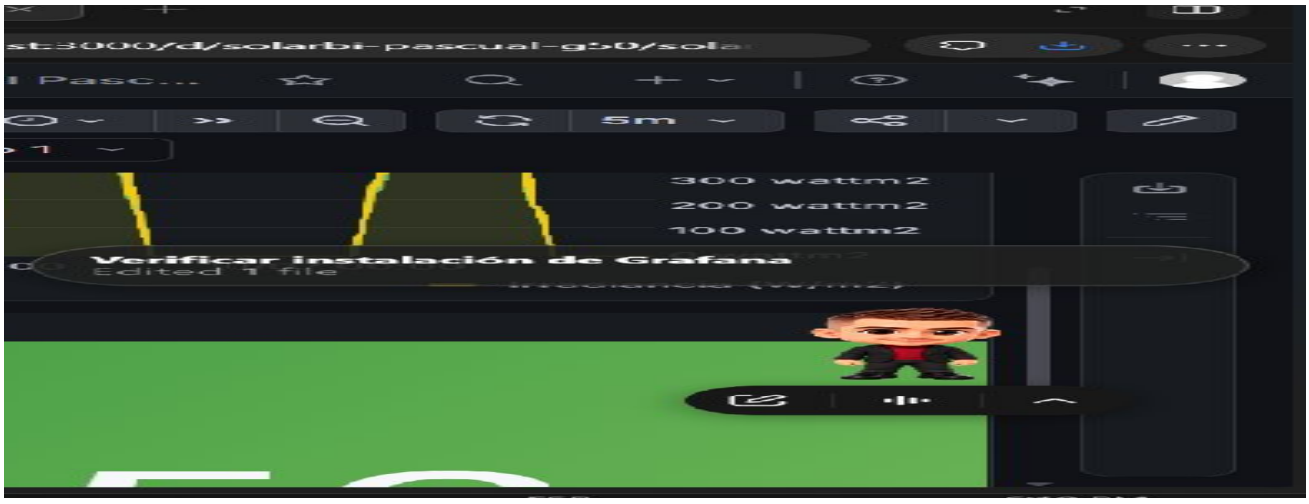
El informe de una página carga PostgreSQL y usa medidas DAX para energía, yield y ahorro. El parámetro de tarifa permite escoger entre 700 y 1.000 COP/kWh; se tomó 800 COP/kWh como supuesto académico configurable, no como tarifa regulada universal.

Medidas principales: **Energía total (kWh)** = SUM(energia_kwh); **Yield** = Energía / potencia nominal; **Tarifa COP** = SELECTEDVALUE(ParametroTarifa[Value], 800); **Ahorro** = Energía × Tarifa.

Paso 5 · Grafana



El panel de series usa *$__timeFilter(ts)* sobre *silver.lectura_5min* y filtra por la variable de dispositivo. Se observan tres curvas solares diarias con potencia e irradiancia reales.



El indicador de calidad consulta *dwh.fact_energia_dia* y muestra 95,56 %. La variable se alimenta desde *dwh.dim_dispositivo*. El data source fue verificado con conexión correcta al mismo PostgreSQL usado por Power BI.

Paso 6 · Reflexión

Grafana resolvió mejor la observación operativa de la telemetría y el refresco frecuente. Power BI resolvió mejor las medidas de negocio, el parámetro de tarifa y la lectura ejecutiva. Lo más costoso fue mantener coherentes conexión, tipos y granularidades entre herramientas. Para un operador usaría Grafana; para un directivo, Power BI. Mantendría PostgreSQL Gold como referencia gobernada y conservaría Silver para investigar anomalías.

Parte C · Repositorio y verificación

Repositorio público: **https://github.com/savelasquezs/Grafana_power_bi**

Elemento	Resultado
Autor	Santiago Velásquez Serna · 1038414022
Modalidad	Trabajo individual; no aplica segundo colaborador
Rama de trabajo	entrega-solarbi
ETL	Ejecutable con python etl/run_etl.py
Grafana	grafana/dashboard.json importable
Power BI	powerbi/SolarBI_Pascual.pbix
Seguridad	Sin contraseñas ni cadenas de conexión versionadas

La entrega individual explica por qué el historial no tiene dos autores. Los cambios se dividieron en commits de ETL, Grafana, Power BI/documentación y entrega final.

Referencias

[1] Grafana Labs. Create dashboards y conceptos de data sources. https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/create-dashboard/

[2] Microsoft Learn. DirectQuery in Power BI. https://learn.microsoft.com/power-bi/connect-data/desktop-directquery-about

[3] Microsoft Learn. Data refresh in Power BI. https://learn.microsoft.com/power-bi/connect-data/refresh-data

[4] Grafana Labs. PostgreSQL template variables. https://grafana.com/docs/grafana/latest/datasources/postgres/template-variables/

[5] DAMA International. DAMA-DMBOK. https://www.dama.org/cpages/body-of-knowledge

[6] Colombia. Ley 1581 de 2012, protección de datos personales. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

[7] Apache Airflow. Cron & Time Intervals. https://airflow.apache.org/docs/apache-airflow/stable/authoring-and-scheduling/cron.html

[8] Microsoft Learn. Incremental refresh overview. https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview

[9] OASIS. MQTT Version 5.0. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[10] Microsoft Learn. Power BI Desktop projects (PBIP). https://learn.microsoft.com/power-bi/developer/projects/projects-overview